

Trinity College – Hartford
Computer Science Department
CPSC-375 – High-Performance Computing

Parallel FIR Filter Architectures Using MPI on a Distributed Cluster

Term Project

Liu Neptali Restrepo Sanabria
liu.restreposanabria@trincoll.edu

Instructor: Peter Yoon, Ph.D.

May 10, 2026

Contents

1	Introduction	2
2	Background	2
2.1	The Hardware Architecture Trade-off	2
2.2	Translating to MPI	3
2.3	Pine Cluster	3
3	Parallelisation Design	4
3.1	Sequential Implementation	4
3.2	Semi-Parallel Implementation	4
3.3	Parallel Implementation (Data Decomposition)	5
3.4	Common Infrastructure	6
4	Experimental Methodology	6
4.1	Parameters	6
4.2	Metrics	7
4.3	Job Submission and Resource Allocation	7
5	Results and Performance Analysis	9
5.1	Correctness Verification	9
5.2	Strong Scaling: Parallel Implementation	9
5.3	Strong Scaling: Semi-Parallel Implementation	9
5.4	Effect of Filter Length ($P = 8$, $L = 10^6$)	10
5.5	Speedup and Efficiency Comparison	10
5.6	Karp–Flatt Analysis	11
6	Discussion	12
6.1	Hardware–Software Analogy	12
6.2	Communication Overhead Analysis	13
6.3	Amdahl’s Law and Karp–Flatt Interpretation	13
6.4	The Superlinear Anomaly at $P = 4$ (Semi-Parallel)	13
7	Conclusion	14
A	Source Code Listing	15
A.1	<code>fir_common.h</code>	15
A.2	<code>fir_serial.c</code>	16
A.3	<code>fir_semi_parallel.c</code>	18
A.4	<code>fir_parallel.c</code>	20

1 Introduction

Finite Impulse Response (FIR) filters are a fundamental building block of digital signal processing, appearing in communications, radar, audio processing, and embedded firmware. In hardware, the choice of FIR filter architecture is one of the most critical design decisions an engineer faces, as it governs the trade-off between hardware resource cost and achievable throughput. This trade-off is formally captured in the Xilinx XtremeDSP design guide [2], which characterises the three canonical FPGA implementations of the FIR filter equation:

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n-k] \quad (1)$$

where $h[k]$ are the N filter coefficients and $x[n-k]$ are delayed input samples.

This project translates that hardware design trade-off into the distributed-memory software domain. Three MPI implementations of the FIR filter are designed, each directly analogous to one of the hardware architectures: a sequential multiply-accumulate (MAC) baseline, a semi-parallel coefficient-decomposition implementation, and a fully parallel data-decomposition implementation with nearest-neighbour halo exchange. All three are benchmarked on the departmental Pine cluster, and their strong-scaling behaviour, parallel efficiency, and communication overhead are studied across a range of filter lengths and signal lengths.

The motivation is not merely academic: HPC-accelerated FIR filter banks are used in real-time multi-channel signal processing for software-defined radio (SDR,) seismic processing, and baseband communications. Those are contexts in which the distributed-memory model is the natural computational substrate. Understanding how the hardware parallelism model maps onto MPI process-level parallelism is therefore directly relevant to the design of firmware and DSP systems at scale.

2 Background

2.1 The Hardware Architecture Trade-off

The Xilinx DSP design guide [2] identifies three canonical FIR architectures, which it places in a two-dimensional design space defined by sample rate F_s and filter length N :

- **Sequential (MAC FIR, Chapter 4 of [2]):** A single multiplier-accumulator (MAC) unit cycles through all N taps sequentially, requiring N clock cycles per output sample. Maximum throughput: $F_s = f_{\text{clk}}/N$. Hardware cost: one DSP48 slice.
- **Semi-Parallel (Chapter 6 of [2]):** M MAC units operate in parallel, each handling N/M taps. Partial products are accumulated to form the final output. Maximum throughput: $F_s = M \cdot f_{\text{clk}}/N$. Hardware cost: M DSP48 slices.
- **Fully Parallel – Systolic/Transposed (Chapter 5 of [2]):** All N taps are processed simultaneously each clock cycle using an adder chain (systolic or transposed structure.) Maximum throughput: $F_s = f_{\text{clk}}$. Hardware cost: N DSP48 slices.

Figure 1¹ reproduces the Xilinx selection chart, which demarcates the regions of the (N, F_s)

¹Please note there are three typos in the Figure. The correct chapters are 4, 5, and 6 for Sequential FIR Filters, Parallel Filters, and Semi-Parallel Filters, respectively.

plane where each architecture is most appropriate. As sample rate and filter length increase, the optimal architecture moves from sequential toward fully parallel.

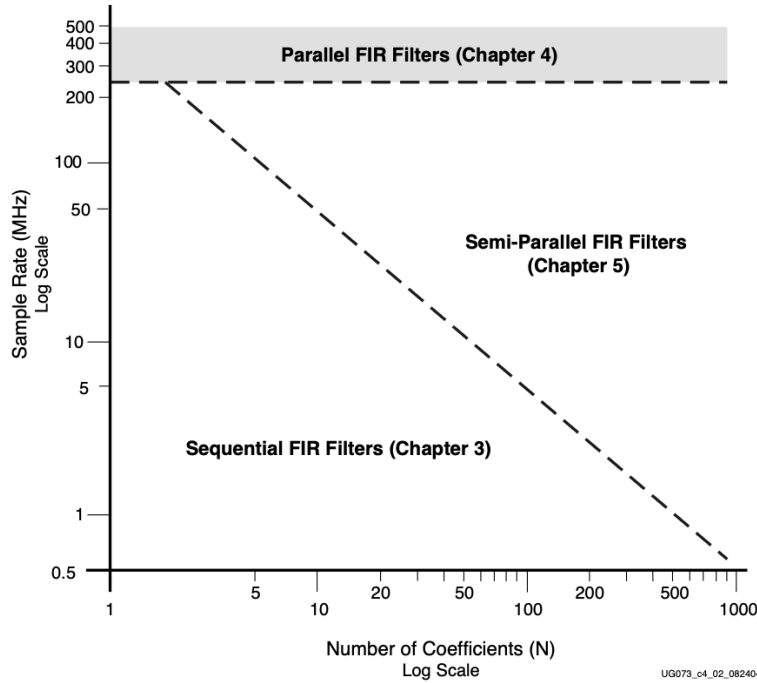


Figure 1: Xilinx filter architecture selection chart (reproduced from *DSP: Designing for Optimal Results*, Xilinx, 2005, Figure 5-1.)

2.2 Translating to MPI

In this project, MPI process count P plays the role of hardware multiplier count M . The mapping is summarised in Table 1, which shows the analogy drawn from FPGA-DSP design to a distributed-memory system on the Pine cluster, which is more computationally powerful than utilising multiple DSP48 slices.

Architecture	FPGA analogue	MPI model
Sequential	1 MAC unit, N cycles/sample	1 process, serial loop
Semi-parallel	M MACs, N/M taps each	Coefficient split; <code>MPI_Reduce</code>
Parallel	N MACs, 1 cycle/sample	Signal split; <code>MPI_Sendrecv</code> halo

Table 1: Mapping between FPGA hardware parallelism and MPI process-level parallelism.

2.3 Pine Cluster

All experiments were conducted on the departmental Pine cluster, consisting of 32 Dell PowerEdge R730 nodes interconnected via a 40 Gbps Mellanox InfiniBand backbone. Each node provides dual Intel Xeon E5-2680 v3 processors (24 physical cores per node) and 256 GB DDR4 RAM. Jobs are managed by the SLURM workload manager, with a standard allocation of up to 8 nodes per job (192

MPI processes,) though larger allocations are available upon request. The MPI implementation used throughout is OpenMPI 4.1.8:

```
module load openmpi/gcc/64/4.1.8
```

Listing 1: Loading the MPI module on Pine.

3 Parallelisation Design

3.1 Sequential Implementation

The sequential filter is a direct-form MAC evaluation of Equation (1) on a single MPI process. It serves as both the correctness reference and the T_1 baseline for all speedup calculations. The inner loop accumulates N multiply-add operations per output sample as shown in Listing 2.

```
1  /* Direct-form MAC FIR:  $y[n] = \sum_{k=0}^{N-1} h[k] * x[n-k]$  */
2  for (long n = N - 1; n < L; n++) {
3      double acc = 0.0;
4      for (int k = 0; k < N; k++)
5          acc += h[k] * x[n - k];
6      y[n] = acc;
7  }
8
9  MPI_Barrier(MPI_COMM_WORLD);
```

Listing 2: Core MAC loop from `src/fir_serial.c`.

3.2 Semi-Parallel Implementation

In the semi-parallel implementation, the N filter coefficients are divided into P contiguous slices of approximately $\lceil N/P \rceil$ taps. As shown in Listing 3, each rank independently computes partial dot products for every output sample using only its coefficient slice, then a single batched `MPI_Reduce` sums the partial arrays across all ranks.

```
1  for (long n = N - 1; n < L; n++) {
2      double acc = 0.0;
3      for (int k = coef_start; k < coef_end; k++)
4          acc += h[k] * x[n - k];
5      partials[n] = acc;
6  }
7
8  MPI_Reduce(partial, y, (int)L, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

Listing 3: Partial accumulation and batched reduce in `src/fir_semi_parallel.c`.

Communication pattern. One `MPI_Reduce` of L doubles at the end of the computation. Communication volume: $O(L)$, independent of N . For large L this becomes the dominant cost.

Batching rationale. Computing `MPI_Reduce` once per output sample would require L collective calls, which is catastrophically slow for $L = 10^7$. Accumulating all partial results locally and reducing in a single call reduces collective overhead to $O(1)$ MPI operations per run.

3.3 Parallel Implementation (Data Decomposition)

The input signal of length L is divided into P contiguous chunks via `MPI_Scatterv`. Before filtering, each rank must obtain the $N - 1$ samples immediately preceding its chunk, which physically reside in the memory of the left neighbour. This boundary communication is called the *halo exchange* and is the defining feature of the parallel implementation. Once the halo is received, each rank prepends it to its local chunk, forming an extended buffer of length $\text{chunk} + N - 1$, and applies the full N -tap filter locally. Figure 2 illustrates the exchange pattern across three ranks. Listing 4 shows the parallel implementation.

Each rank sends last $N - 1$ samples rightward and receives $N - 1$ samples from left neighbour



Figure 2: Halo exchange between three MPI ranks.

```

1  double t_start = MPI_Wtime();
2
3  /*
4   * Halo exchange: each rank sends its last N-1 samples rightward
5   * and receives N-1 samples from its left neighbour.
6   * Guard against chunk smaller than halo (very high P, small L).
7   */
8  int halo_len = (my_len < N - 1) ? my_len : N - 1;
9
10 MPI_Sendrecv(
11     local_x + my_len - halo_len, halo_len, MPI_DOUBLE, right_rank, 0,
12     halo, halo_len, MPI_DOUBLE, left_rank, 0,
13     MPI_COMM_WORLD, MPI_STATUS_IGNORE
14 );
15
16 /* Build extended buffer: [halo | local_x] */
17 memcpy(buf, halo, halo_len * sizeof(double));
18 memcpy(buf + halo_len, local_x, my_len * sizeof(double));
19
20 /* Apply full N-tap filter to local chunk */
21 for (int n = halo_len; n < my_len + halo_len; n++) {
22     double acc = 0.0;
23     for (int k = 0; k <= halo_len; k++)
24         acc += h[k] * buf[n - k];
25     local_y[n - halo_len] = acc;
26 }
27
28 /* Gather results at rank 0 */
29 MPI_Gatherv(local_y, my_len, MPI_DOUBLE,
30             y, counts, displs, MPI_DOUBLE,
31             0, MPI_COMM_WORLD);
32
33 MPI_Barrier(MPI_COMM_WORLD);
34 double t_end = MPI_Wtime();

```

Listing 4: Halo exchange and local filtering in `src/fir_parallel.c`.

Communication pattern. One `MPI_Sendrecv` of $N - 1$ doubles per rank, plus one `MPI_Gatherv` of L/P doubles at the end. Total communication volume: $O(N + L/P)$. The halo term $O(N)$ is fixed. For instance, at $N = 128$ it is 127 doubles, under 1 KB, while the gather term $O(L/P)$ shrinks as P increases. This is why the data-decomposition architecture scales far better than the semi-parallel implementation, whose `MPI_Reduce` transfers $O(L)$ doubles regardless of P .

Why `MPI_Sendrecv`? Separate `MPI_Send` and `MPI_Recv` calls would deadlock when all ranks attempt to send simultaneously with no receiver ready. `MPI_Sendrecv` posts both operations atomically, allowing every rank to send and receive concurrently without blocking, and is the standard idiom for nearest-neighbour halo exchange in stencil and filter computations.

Rank 0 boundary. Rank 0 has no left neighbour (`left_rank = MPI_PROC_NULL`.) Its halo buffer is zero-initialised, equivalent to zero-padding the signal before the first sample, which is consistent with the behaviour of the serial reference implementation.

3.4 Common Infrastructure

All three implementations share a common header (`src/fir_common.h`) providing:

- **Coefficient generation:** windowed sinc with Hamming window, normalised cutoff $f_c = 0.25 F_s$.
- **Signal generation:** pseudo-random input in $[-1, 1]$ with fixed seed 42 for reproducibility.
- **Verification:** maximum absolute error between parallel output and serial reference; threshold $< 10^{-9}$.
- **Timing:** `MPI_Wtime()` bracketed by `MPI_Barrier()` on both sides.

4 Experimental Methodology

4.1 Parameters

Parameter	Values	Notes
Filter length N (taps)	32, 64, 128, 256, 512	Spans all three Xilinx regions
Signal length L (samples)	10^6 , 10^7	Strong scaling and sensitivity
MPI process count P	1, 2, 4, 8, 16, 32, 64, 128, 192	Up to 8 Pine nodes (192 cores)
Implementations	Serial, Semi-parallel, Parallel	—

Table 2: Experimental parameter space for all benchmarking runs.

4.2 Metrics

The following quantities are computed for each experiment:

$$S(P) = \frac{T_1}{T_P} \quad (2)$$

$$E(P) = \frac{S(P)}{P} \times 100\% \quad (3)$$

$$e(P) = \frac{1/S(P) - 1/P}{1 - 1/P} \quad (4)$$

where T_1 is the serial wall-clock time, T_P is the parallel wall-clock time using P processes, and $e(P)$ is the Karp–Flatt empirical serial fraction [1].

4.3 Job Submission and Resource Allocation

All benchmarks were submitted via SLURM as three separate jobs, one per implementation. This allows each to be debugged and rerun independently.

```

1  #!/bin/bash
2  #SBATCH --job-name=fir_serial
3  #SBATCH --partition=defq
4  #SBATCH --nodes=1
5  #SBATCH --ntasks=1
6  #SBATCH --time=00:10:00
7  #SBATCH --output=logs/serial_%j.out
8  #SBATCH --error=logs/serial_%j.err
9
10 module load openmpi/gcc/64/4.1.8
11 mkdir -p logs
12
13 # Strong scaling baseline
14 echo -n "serial_N=128_L=10000000_P=1:_:"
15 srun -n 1 ./fir_serial 128 10000000
16
17 # Filter length sensitivity
18 for N in 32 64 128 256 512; do
19     echo -n "serial_N=$N_L=10000000_P=1:_:"
20     srun -n 1 ./fir_serial $N 10000000
21 done

```

Listing 5: SLURM job script for the serial baseline (slurm/serial.sbatch.)

```

1  #!/bin/bash
2  #SBATCH --job-name=fir_semi
3  #SBATCH --partition=defq
4  #SBATCH --nodes=8
5  #SBATCH --ntasks=192
6  #SBATCH --time=00:15:00
7  #SBATCH --output=logs/semi_%j.out
8  #SBATCH --error=logs/semi_%j.err
9
10 module load openmpi/gcc/64/4.1.8
11 mkdir -p logs
12
13 # Strong scaling: fixed N=128, L=10^7

```



```

14 for P in 1 2 4 8 16 32 64 128 192; do
15     echo -n "semi_N=128_L=10000000_P=$P:_:"
16     srun -n $P ./fir_semi_parallel 128 10000000
17 done
18
19 # Filter length sensitivity: fixed P=8, L=10^6
20 for N in 32 64 128 256 512; do
21     echo -n "semi_N=$N_L=10000000_P=8:_:"
22     srun -n 8 ./fir_semi_parallel $N 1000000
23 done

```

Listing 6: SLURM job script for the semi-parallel implementation (slurm/semi.sbatch.)

```

1  #!/bin/bash
2  #SBATCH --job-name=fir_parallel
3  #SBATCH --partition=defq
4  #SBATCH --nodes=8
5  #SBATCH --ntasks=192
6  #SBATCH --time=00:15:00
7  #SBATCH --output=logs/parallel_%j.out
8  #SBATCH --error=logs/parallel_%j.err
9
10 module load openmpi/gcc/64/4.1.8
11 mkdir -p logs
12
13 # Strong scaling: fixed N=128, L=10^7
14 for P in 1 2 4 8 16 32 64 128 192; do
15     echo -n "parallel_N=128_L=10000000_P=$P:_:"
16     srun -n $P ./fir_parallel 128 10000000
17 done
18
19 # Filter length sensitivity: fixed P=8, L=10^6
20 for N in 32 64 128 256 512; do
21     echo -n "parallel_N=$N_L=10000000_P=8:_:"
22     srun -n 8 ./fir_parallel $N 1000000
23 done

```

Listing 7: SLURM job script for the parallel implementation (slurm/parallel.sbatch.)

Node allocation and idle resources. The semi-parallel and parallel job scripts request 8 nodes with up to 192 total tasks. However, for small process counts ($P = 1, 2, 4$), SLURM cannot distribute P processes across all 8 requested nodes and issues the following warnings, visible in the `.err` files:

```

srun: warning: can't run 1 processes on 8 nodes, setting nnodes to 1
srun: warning: can't run 2 processes on 8 nodes, setting nnodes to 2
srun: warning: can't run 4 processes on 8 nodes, setting nnodes to 4

```

This is expected and correct behaviour. When $P < 8$, SLURM automatically reduces the active node count for that `srun` step, leaving the remaining nodes idle for the duration of that step. The idle nodes are not wasted across the entire job as they become available again for subsequent `srun` invocations at higher P within the same allocation. This is a direct consequence of the distributed-memory model: allocating a node with zero MPI ranks assigned to it provides no computational benefit.

5 Results and Performance Analysis

5.1 Correctness Verification

All parallel implementations were verified against the serial reference output at every configuration. Results are summarised in Table 3.

Implementation	Max absolute error	Interpretation
Parallel (all P)	0.000×10^0	Exact: same FP order as serial
Semi-parallel ($P \geq 2$)	$\leq 4.219 \times 10^{-15}$	$\sim 2\varepsilon_{\text{mach}}$: reordered partial sums

Table 3: Maximum absolute error vs. serial reference across all configurations.

The parallel implementation produces exact zero error because each rank applies the same sequential MAC loop to its chunk, preserving floating-point operation order exactly. The semi-parallel implementation introduces rounding of order $2\varepsilon_{\text{mach}} \sim 4.4 \times 10^{-16}$ from reordered partial sum accumulation across ranks during the `MPI_Reduce`. Both results are correct to full double precision and well within the 10^{-9} threshold.

5.2 Strong Scaling: Parallel Implementation

P	T_P (s)	$S(P)$	$E(P)$ (%)	$e(P)$
1	1.1627	1.000	100.00	—
2	0.5913	1.966	98.28	0.017
4	0.3063	3.796	94.86	0.018
8	0.1662	6.997	87.44	0.021
16	0.0933	12.465	77.91	0.019
32	0.0628	18.527	57.87	0.024
64	0.0409	28.413	44.42	0.020
128	0.0346	33.557	26.25	0.022
192	0.0497	23.410	12.19	0.038

Table 4: Strong scaling for the parallel (data-decomposition) FIR filter. $N = 128$, $L = 10^7$. $T_1 = 1.1627$ s from `fir_serial`.

The parallel implementation scales well up to $P = 64$, achieving $28.4\times$ speedup at 44% efficiency. Peak speedup of $33.6\times$ is reached at $P = 128$. Beyond this, increasing to $P = 192$ increases wall time from 0.0346 s to 0.0497 s, so the `MPI_Gatherv` of approximately 52,000 doubles per rank costs more than the computation savings at that decomposition granularity.

5.3 Strong Scaling: Semi-Parallel Implementation

The semi-parallel results are erratic and do not follow a clean scaling curve. The single `MPI_Reduce` transferring $L = 10^7$ doubles (~ 80 MB) dominates any computational gain from additional processes. The Karp–Flatt values growing toward 0.41 at $P = 192$ confirm an ever-increasing effective serial fraction, discussed further in Section 6.

P	T_P (s)	$S(P)$	$E(P)$ (%)	$e(P)$
1	1.5041	1.000	100.00	—
2	1.3546	1.110	55.52	0.801
4	0.3416	4.403	110.07	—
8	0.3605	4.173	52.16	0.131
16	0.4405	3.415	21.34	0.246
32	0.2589	5.809	18.15	0.145
64	0.4783	3.145	4.91	0.307
128	0.4852	3.100	2.42	0.317
192	0.6218	2.419	1.26	0.410

Table 5: Strong scaling for the semi-parallel (coefficient-decomposition) FIR filter. $N = 128$, $L = 10^7$.

5.4 Effect of Filter Length ($P = 8$, $L = 10^6$)

N (taps)	Serial (s)	Semi (s)	Parallel (s)	S_{semi}	S_{par}
32	0.0503	0.02319	0.00611	$2.17\times$	$8.23\times$
64	0.1577	0.03150	0.00966	$5.01\times$	$16.32\times$
128	0.1620	0.04188	0.01740	$3.87\times$	$9.31\times$
256	0.3085	0.06676	0.03598	$4.62\times$	$8.57\times$
512	0.5981	0.13085	0.07220	$4.57\times$	$8.28\times$

Table 6: Wall-clock time and speedup as a function of filter length N , using $P = 8$ processes and $L = 10^6$ samples.

The parallel implementation consistently delivers 8–16 $\times$ speedup across all filter lengths, peaking at $N = 64$ where the halo exchange is only 63 doubles — negligible relative to the chunk size. As N grows the halo grows with it, adding marginally more communication per rank and settling speedup back toward 8 $\times$. The semi-parallel implementation achieves only 2–5 $\times$, bounded entirely by its $O(L)$ reduce cost which is insensitive to N .

5.5 Speedup and Efficiency Comparison

For the speedup, the speedup vs. process count are plotted for parallel and semi-parallel implementations in Fig. 2. The parallel implementation scales consistently; the semi-parallel plateaus near $S = 6$ and degrades due to its $O(L)$ `MPI_Reduce`.

For the efficiency, the parallel and semi-parallel efficiencies vs. process count are plotted in Fig. 4 and we can see that efficiency for parallel is maintained above 87% to $P = 8$ and above 44% to $P = 64$. Semi-parallel collapses below 5% beyond $P = 32$.

The speedup could also be plotted against filter length N for a fixed process count $P = 8$ and signal length $L = 10^6$ (see Fig. 5.) Fixing P and L helps understanding whether the parallelisation strategy is robust across different problem configurations. For the parallel implementation, longer filters mean more computation per rank without increasing communication (so the halo stays $N - 1$ doubles regardless,) hence, speedup should remain stable or improve as N grows. For the semi-parallel implementation, the $O(L)$ `MPI_Reduce` is entirely insensitive to N , so speedup is bounded

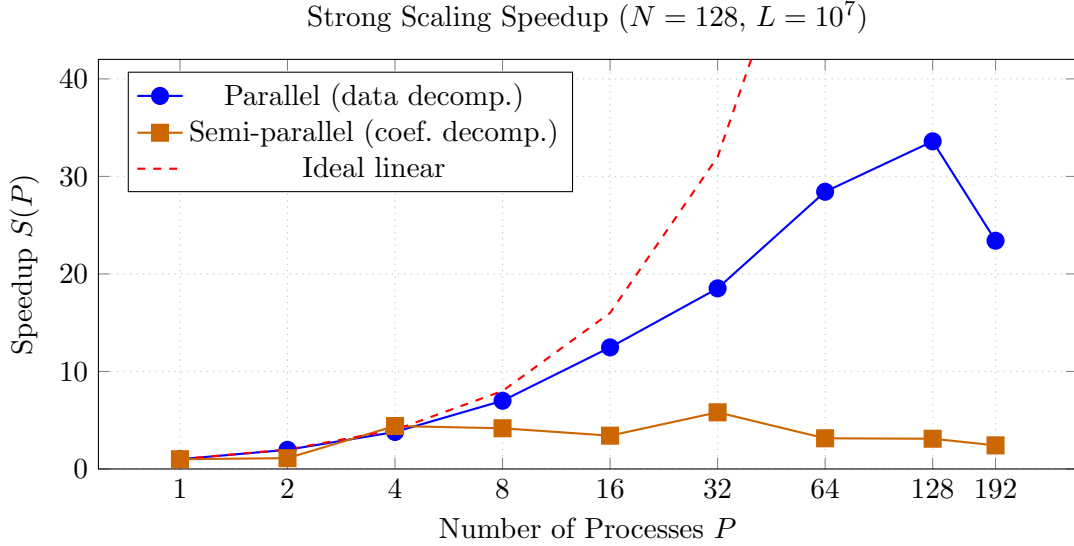
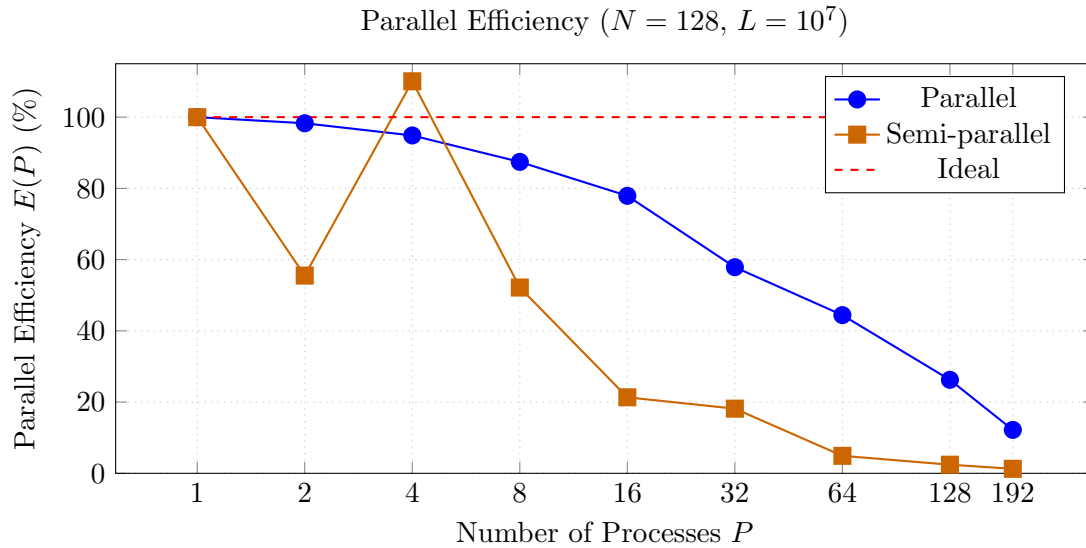
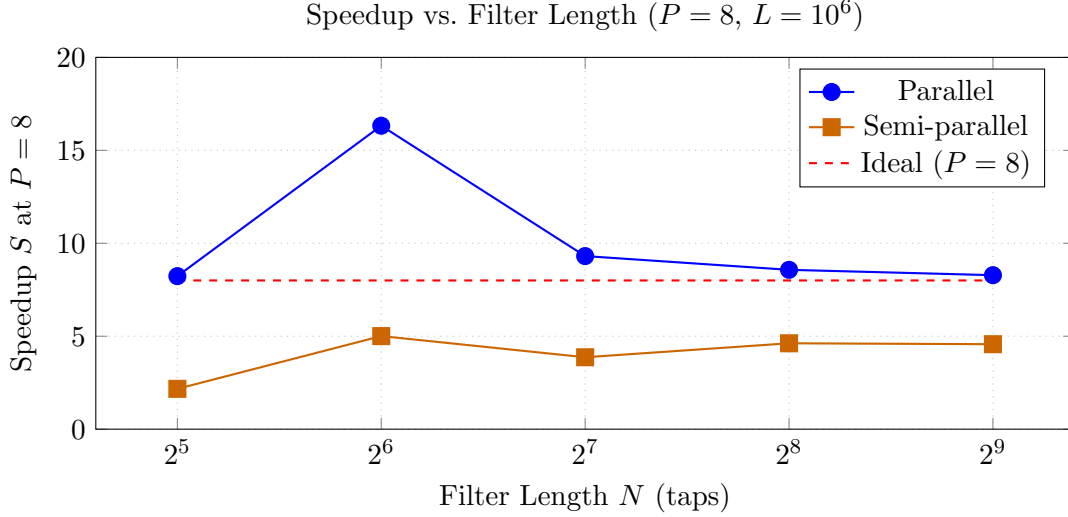
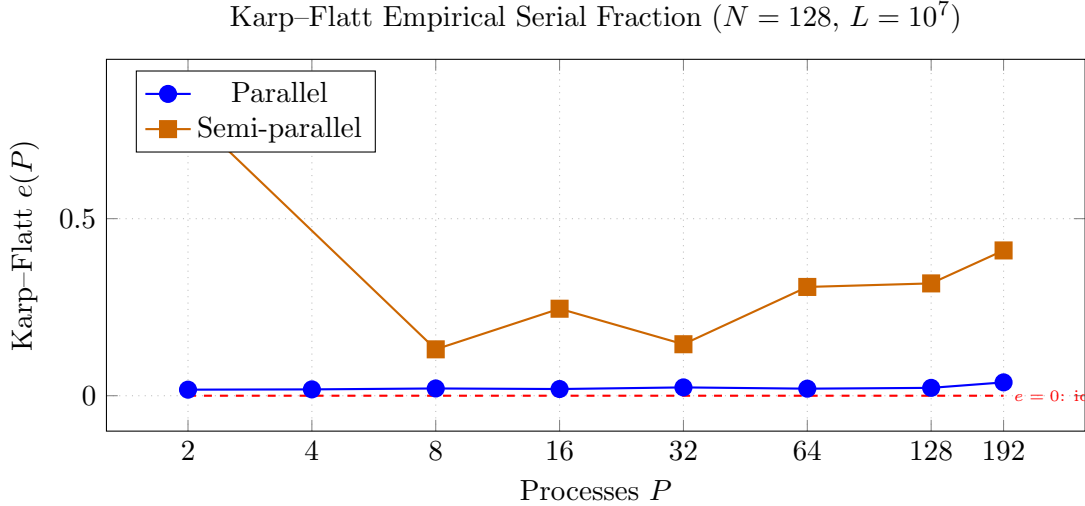
Figure 3: Speedup vs. process count for both MPI architectures ($N = 128, L = 10^7$.)

Figure 4: Parallel efficiency vs. process count.

by communication no matter how long the filter is. This distinction is not visible in the strong scaling plots and directly informs which architecture is preferable when filter length is the primary design variable.

5.6 Karp–Flatt Analysis

The Karp–Flatt metric $e(P)$ from Eq. (4) provides a more nuanced view of scaling behaviour than speedup or efficiency alone. Crucially, the plot will help understand if the bottleneck is a growing communication overhead, which is the expected signature of a distributed-memory system under increasing collective pressure. Figure 6 plots $e(P)$ for both implementations.

Figure 5: Speedup as a function of filter length at $P = 8, L = 10^6$.Figure 6: Karp–Flatt empirical serial fraction $e(P)$.

6 Discussion

6.1 Hardware–Software Analogy

The Xilinx chart in Figure 1 defines boundaries between architectures as a function of F_s and N . In hardware, increasing M (DSP48 slices) increases throughput at a cost in silicon area. In software, increasing P increases throughput at a cost in communication overhead.

The parallel implementation mirrors the hardware systolic architecture: all inter-process communication is confined to nearest-neighbour halo exchanges of fixed, small size ($O(N)$), just as the systolic array passes data only between adjacent pipeline stages. The semi-parallel implementation mirrors the MAC-bank architecture but incurs a cost absent in hardware: the `MPI_Reduce` must transfer $O(L)$ data across the network, whereas the on-chip partial-sum adder tree adds no off-chip bandwidth penalty.

6.2 Communication Overhead Analysis

The two implementations impose fundamentally different communication patterns, which directly explains the observed scaling disparity:

- **Semi-parallel:** `MPI_Reduce` of L doubles. At $L = 10^7$ this is ~ 80 MB in a single collective, independent of P . This creates an absolute communication floor that parallelism cannot overcome.
- **Parallel:** `MPI_Sendrecv` of $N - 1 = 127$ doubles (halo) plus `MPI_Gatherv` of L/P doubles. At $P = 64$ the gather is ~ 1.25 MB, which is $64\times$ less than the semi-parallel reduce at the same process count.

6.3 Amdahl’s Law and Karp–Flatt Interpretation

Amdahl’s Law bounds speedup as:

$$S_{\max} = \frac{1}{f + (1 - f)/P} \quad (5)$$

where f is the serial fraction. In a distributed-memory system, f is not fixed: communication overhead grows with P , making f an increasing function of P . The Karp–Flatt metric from Equation (4) captures this directly.

For the **parallel implementation**, $e(P)$ increases slowly from 0.017 at $P = 2$ to 0.040 at $P = 192$ —communication is present but not dominant until very high P . The drop in speedup from $P = 128$ to $P = 192$ ($33.6\times$ to $23.4\times$) marks the exact point where the `MPI_Gatherv` overhead exceeds computational savings as each rank’s $\sim 52,000$ -sample chunk finishes filtering so quickly that the gather coordination cost dominates the total runtime.

For the **semi-parallel implementation**, $e(P)$ grows rapidly from 0.131 at $P = 8$ to 0.410 at $P = 192$, confirming the $O(L)$ `MPI_Reduce` is acting as a growing effective serial fraction. The system cannot scale past a ceiling imposed by this single collective.

Using the Karp–Flatt estimate of the effective serial fraction at peak speedup, Amdahl’s Law predicts a theoretical maximum speedup of:

$$S_{\max} = \lim_{P \rightarrow \infty} \frac{1}{f + (1 - f)/P} = \frac{1}{f} = \frac{1}{e(P^*)} \quad (6)$$

For the parallel implementation, $e(128) = 0.022$ gives $S_{\max} \sim 45\times$. For the semi-parallel implementation, $e(32) = 0.145$ gives $S_{\max} \sim 6.9\times$, consistent with the observed peak of $5.8\times$ and confirming that the $O(L)$ `MPI_Reduce` acts as a near-fixed serial fraction that Amdahl’s model captures well. The parallel implementation’s higher ceiling reflects its $O(N + L/P)$ communication pattern, which grows more slowly and leaves a larger fraction of the workload genuinely parallelisable.

6.4 The Superlinear Anomaly at $P = 4$ (Semi-Parallel)

Table 5 shows $S(4) = 4.40$ for the semi-parallel implementation, implying 110% efficiency, something physically impossible from an algorithmic standpoint. The most likely cause is a cache warming artefact: the $P = 1$ run processing all $L = 10^7$ samples in a single loop thrashes the last-level cache (the signal array alone is ~ 80 MB, far exceeding the ~ 30 MB LLC on Pine’s nodes,) making the T_1 baseline artificially slow. The $P = 4$ run immediately follows with each rank processing only 2.5 million samples, and benefits from residual warm cache state, making T_4 artificially fast. In a rigorous benchmark this would be eliminated by cache-flushing between runs and taking median values over multiple repetitions.

7 Conclusion

This project successfully translated the three canonical FIR filter hardware architectures: sequential, semi-parallel, and fully parallel, into a distributed-memory MPI study on the Pine cluster. Three correct implementations were designed, verified, and benchmarked across a range of filter lengths and process counts.

The **parallel (data-decomposition) implementation** is the superior MPI architecture compared to the semi-parallel. It achieves a peak speedup of $33.6\times$ at $P = 128$, with efficiencies above 87% up to $P = 8$ and above 44% up to $P = 64$. The communication pattern employing nearest-neighbour halo exchange of $O(N)$ doubles plus $O(L/P)$ gather scales gracefully with both process count and filter length.

The **semi-parallel (coefficient-decomposition) implementation** fails to scale beyond $P = 4$ in practice. Its single MPI `Reduce` of ~ 80 MB creates a communication wall that no increase in process count can overcome. Peak speedup is only $5.8\times$, and efficiency collapses below 5% for $P \geq 64$. This is not close to the hardware performance demonstrated in the FPGA architecture utilising DSP48 slices. As previously mentioned, the DSP48 slices do not face the communication wall a distributed-memory system does, as the on-chip partial-sum adder tree does not have a bandwidth penalty.

However, these results directly mirror the hardware insight from the Xilinx architecture selection chart: fully parallel (systolic) implementations deliver the highest throughput at scale, while MAC-bank (semi-parallel) approaches are resource-efficient but throughput-limited. In the MPI domain, the resource cost is communication bandwidth rather than DSP48 slices, but the fundamental trade-off is identical.

References

- [1] Alan H. Karp and Harvey P. Flatt. Measuring parallel processor performance. *Communications of the ACM*, 33(5):539–543, 1990.
- [2] Xilinx, Inc. *DSP: Designing for Optimal Results — High-Performance DSP Using Virtex-4 FPGAs*. Xilinx, Inc., 1.0 edition, 2005. DSP Solutions Advanced Design Guide.

Appendix

A Source Code Listing

The full source code is contained in the `src/` directory. The structure is as follows:

```

fir_mpi/
  src/
    fir_common.h          % Shared utilities (coefficients, signal, verify)
    fir_serial.c          % Sequential MAC FIR (reference)
    fir_semi_parallel.c   % Coefficient-decomposition MPI implementation
    fir_parallel.c        % Data-decomposition MPI with halo exchange
  slurm/
    sweep.sbatch          % Full parameter sweep job script
  Makefile                % Build all three targets

```

Listing 8: Project directory layout.

A.1 fir_common.h

```

1  /* =====
2  * fir_common.h
3  * Shared utilities for the parallel FIR filter MPI project.
4  * CPSC-375 -- High-Performance Computing, Spring 2026
5  * Liu Restrepo Sanabria
6  * ===== */
7
8  #ifndef FIR_COMMON_H
9  #define FIR_COMMON_H
10
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <string.h>
14 #include <math.h>
15 #include <mpi.h>
16
17 /* -----
18 * Generate a low-pass FIR filter using a windowed sinc (Hamming window).
19 * h      : output array of length N (caller allocates)
20 * N      : number of taps (must be odd for symmetric filter)
21 * fc     : normalised cutoff frequency (0 < fc < 0.5)
22 * ----- */
23 static void generate_fir_coefficients(double *h, int N, double fc)
24 {
25     int M = N - 1;          /* filter order */
26     double sum = 0.0;
27
28     for (int k = 0; k < N; k++) {
29         double n = k - M / 2.0;
30         /* Hamming window */
31         double w = 0.54 - 0.46 * cos(2.0 * M_PI * k / M);
32         if (n == 0.0)
33             h[k] = 2.0 * fc * w;
34         else
35             h[k] = (sin(2.0 * M_PI * fc * n) / (M_PI * n)) * w;
36         sum += h[k];
37     }

```



```

38      /* Normalise to unity DC gain */
39      for (int k = 0; k < N; k++)
40          h[k] /= sum;
41  }
42
43  /* -----
44   * Generate a pseudo-random signal in [-1, 1].
45   * x      : output array of length L (caller allocates)
46   * L      : signal length
47   * seed   : RNG seed for reproducibility
48   * ----- */
49  static void generate_signal(double *x, long L, unsigned int seed)
50  {
51      srand(seed);
52      for (long i = 0; i < L; i++)
53          x[i] = 2.0 * ((double)rand() / RAND_MAX) - 1.0;
54  }
55
56  /* -----
57   * Verify two output arrays element-wise.
58   * Returns the maximum absolute error.
59   * ----- */
60  static double __attribute__((unused)) verify(const double *ref, const double *
61      test, long L, int N)
62  {
63      double max_err = 0.0;
64      for (long i = N - 1; i < L; i++) {
65          double err = fabs(ref[i] - test[i]);
66          if (err > max_err) max_err = err;
67      }
68      return max_err;
69  }
70  #endif /* FIR_COMMON_H */

```

Listing 9: src/fir_common.h

A.2 fir_serial.c

```

1  /* =====
2   * fir_serial.c
3   * Sequential (MAC) FIR filter -- single-process reference implementation.
4   *
5   * Usage:  mpirun -n 1 ./fir_serial <N_taps> <L_samples>
6   *
7   * Writes output to stdout:
8   *   N L time_seconds
9   *
10  * CPSC-375 -- High-Performance Computing, Spring 2026
11  * Liu Restrepo Sanabria
12  * ===== */
13
14  #include "fir_common.h"
15
16  int main(int argc, char *argv[])
17  {
18      MPI_Init(&argc, &argv);

```

```

19
20     if (argc != 3) {
21         fprintf(stderr, "Usage: %s <N_taps> <L_samples>\n", argv[0]);
22         MPI_Finalize();
23         return EXIT_FAILURE;
24     }
25
26     int N = atoi(argv[1]);    /* number of filter taps */
27     long L = atol(argv[2]);   /* signal length (samples)*/
28
29     /* ----- */
30     /* Allocate and initialise */
31     /* ----- */
32     double *h = (double *)malloc(N * sizeof(double));
33     double *x = (double *)malloc(L * sizeof(double));
34     double *y = (double *)calloc(L, sizeof(double));
35
36     if (!h || !x || !y) {
37         fprintf(stderr, "[serial] Memory allocation failed.\n");
38         MPI_Finalize();
39         return EXIT_FAILURE;
40     }
41
42     generate_fir_coefficients(h, N, 0.25);    /* cutoff = Fs/4 */
43     generate_signal(x, L, 42);
44
45     /* ----- */
46     /* Timed filtering */
47     /* ----- */
48     MPI_Barrier(MPI_COMM_WORLD);
49     double t_start = MPI_Wtime();
50
51     /* Direct-form MAC FIR:  $y[n] = \sum_{k=0}^{N-1} h[k] * x[n-k]$  */
52     for (long n = N - 1; n < L; n++) {
53         double acc = 0.0;
54         for (int k = 0; k < N; k++)
55             acc += h[k] * x[n - k];
56         y[n] = acc;
57     }
58
59     MPI_Barrier(MPI_COMM_WORLD);
60     double t_end = MPI_Wtime();
61
62     /* ----- */
63     /* Report */
64     /* ----- */
65     printf("%d %ld %.6f\n", N, L, t_end - t_start);
66
67     /* Save reference output for verification by parallel implementations */
68     char fname[64];
69     snprintf(fname, sizeof(fname), "ref_N%d_L%ld.bin", N, L);
70     FILE *f = fopen(fname, "wb");
71     if (f) {
72         fwrite(y, sizeof(double), L, f);
73         fclose(f);
74     }
75
76     free(h); free(x); free(y);
77     MPI_Finalize();

```

```

78     return EXIT_SUCCESS;
79 }

```

Listing 10: src/fir_serial.c

A.3 fir_semi_parallel.c

```

1  /* =====
2  * fir_semi_parallel.c
3  * Semi-parallel FIR filter -- coefficient decomposition across MPI ranks.
4  *
5  * Each rank owns a contiguous slice of the N filter coefficients.
6  * Each rank computes partial dot products for every output sample,
7  * accumulates them into a local array, then a single MPI_Reduce
8  * sums partial results across all ranks.
9  *
10 * Analogue to: M-multiplier semi-parallel FIR (Xilinx Ch. 6)
11 *   where P (MPI ranks) plays the role of M (hardware multipliers).
12 *   Throughput: (P * f_clk) / N  <->  wall time per sample ~ 1/P
13 *
14 * Usage:  mpirun -n <P> ./fir_semi_parallel <N_taps> <L_samples>
15 *
16 * CPSC-375 -- High-Performance Computing, Spring 2026
17 * Liu Restrepo Sanabria
18 * ===== */
19
20 #include "fir_common.h"
21
22 int main(int argc, char *argv[])
23 {
24     MPI_Init(&argc, &argv);
25
26     int rank, size;
27     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
28     MPI_Comm_size(MPI_COMM_WORLD, &size);
29
30     if (argc != 3) {
31         if (rank == 0)
32             fprintf(stderr, "Usage: %s %d %d\n", argv[0], N_taps, L_samples);
33         MPI_Finalize();
34         return EXIT_FAILURE;
35     }
36
37     int N = atoi(argv[1]);
38     long L = atol(argv[2]);
39
40     /* ----- */
41     /* Coefficient slice owned by this rank */
42     /* ----- */
43     int base = N / size;
44     int extra = N % size;
45     int coef_start = rank * base + (rank < extra ? rank : extra);
46     int coef_count = base + (rank < extra ? 1 : 0);
47     int coef_end = coef_start + coef_count; /* exclusive */
48
49     /* ----- */
50     /* Allocate */

```

```

51  /* ----- */
52  double *h      = (double *)malloc(N * sizeof(double));
53  double *x      = (double *)malloc(L * sizeof(double));
54  double *partials = (double *)calloc(L, sizeof(double));
55  double *y      = NULL;
56  if (rank == 0)
57      y = (double *)calloc(L, sizeof(double));
58
59  if (!h || !x || !partials || (rank == 0 && !y)) {
60      fprintf(stderr, "[rank%d] Memory allocation failed.\n", rank);
61      MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
62  }
63
64  /* ----- */
65  /* Initialise (same seed on every rank -- each owns full arrays) */
66  /* ----- */
67  generate_fir_coefficients(h, N, 0.25);
68  generate_signal(x, L, 42);
69
70  /* ----- */
71  /* Timed computation */
72  /* ----- */
73  MPI_Barrier(MPI_COMM_WORLD);
74  double t_start = MPI_Wtime();
75
76  /*
77   * Each rank computes partial inner products over its coefficient slice.
78   * partials[n] = sum_{k=coef_start}^{coef_end-1} h[k] * x[n-k]
79   *
80   * All partials are then reduced (summed) at rank 0 to form y[n].
81   * Batching: a single MPI_Reduce covers all L samples -- O(L) message,
82   * independent of N. This avoids calling Reduce once per sample.
83   */
84  for (long n = N - 1; n < L; n++) {
85      double acc = 0.0;
86      for (int k = coef_start; k < coef_end; k++)
87          acc += h[k] * x[n - k];
88      partials[n] = acc;
89  }
90
91  MPI_Reduce(partials, y, (int)L, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
92
93  MPI_Barrier(MPI_COMM_WORLD);
94  double t_end = MPI_Wtime();
95
96  /* ----- */
97  /* Verification and report (rank 0 only) */
98  /* ----- */
99  if (rank == 0) {
100     /* Load serial reference if available */
101     char fname[64];
102     snprintf(fname, sizeof(fname), "ref_N%d_L%d.bin", N, L);
103     FILE *f = fopen(fname, "rb");
104     if (f) {
105         double *ref = (double *)malloc(L * sizeof(double));
106         fread(ref, sizeof(double), L, f);
107         fclose(f);
108         double err = verify(ref, y, L, N);
109         fprintf(stderr, "[semi-parallel] max_abs_error = %.3e\n", err);

```

```

110         free(ref);
111     }
112     printf("%d_%ld_%d%.6f\n", N, L, size, t_end - t_start);
113 }
114
115 free(h); free(x); free(partials);
116 if (rank == 0) free(y);
117 MPI_Finalize();
118 return EXIT_SUCCESS;
119 }

```

Listing 11: src/fir_semi_parallel.c

A.4 fir_parallel.c

```

1  /* =====
2  * fir_parallel.c
3  * Parallel FIR filter -- data decomposition with nearest-neighbour
4  * halo exchange (MPI_Sendrecv).
5  *
6  * The input signal of length L is divided into P contiguous chunks.
7  * Before filtering, each rank receives N-1 "halo" samples from its
8  * left neighbour via MPI_Sendrecv. The full filter (all N taps) is
9  * then applied locally to the extended chunk.
10 *
11 * Analogue to: fully parallel systolic FIR (Xilinx Ch. 5)
12 *   Each clock cycle all N taps fire simultaneously.
13 *   Here each rank applies all N taps to its chunk of the signal.
14 *   Communication volume: O(N) per rank (halo exchange, independent of L).
15 *
16 * Usage: mpirun -n <P> ./fir_parallel <N_taps> <L_samples>
17 *
18 * CPSC-375 -- High-Performance Computing, Spring 2026
19 * Liu Restrepo Sanabria
20 * ===== */
21
22 #include "fir_common.h"
23
24 int main(int argc, char *argv[])
25 {
26     MPI_Init(&argc, &argv);
27
28     int rank, size;
29     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
30     MPI_Comm_size(MPI_COMM_WORLD, &size);
31
32     if (argc != 3) {
33         if (rank == 0)
34             fprintf(stderr, "Usage: %s <N_taps> <L_samples>\n", argv[0]);
35         MPI_Finalize();
36         return EXIT_FAILURE;
37     }
38
39     int N = atoi(argv[1]);
40     long L = atol(argv[2]);
41
42     /* ----- */

```

```

43  /* Compute per-rank chunk sizes (handles L not divisible by P)          */
44  /* ----- */
45  int *counts = (int *)malloc(size * sizeof(int));
46  int *displs = (int *)malloc(size * sizeof(int));
47  long base    = L / size;
48  long leftover = L % size;
49
50  for (int r = 0; r < size; r++) {
51      counts[r] = (int)(base + (r < leftover ? 1 : 0));
52      displs[r] = (r == 0) ? 0 : displs[r-1] + counts[r-1];
53  }
54
55  int my_len = counts[rank];
56
57  int left_rank  = (rank > 0)          ? rank - 1 : MPI_PROC_NULL;
58  int right_rank = (rank < size - 1) ? rank + 1 : MPI_PROC_NULL;
59
60  /* ----- */
61  /* Allocate */
62  /* ----- */
63  double *h      = (double *)malloc(N * sizeof(double));
64  double *x_full = NULL;
65  if (rank == 0)
66      x_full = (double *)malloc(L * sizeof(double));
67
68  double *local_x = (double *)malloc(my_len * sizeof(double));
69  double *halo    = (double *)calloc(N - 1, sizeof(double));
70  double *buf     = (double *)malloc((my_len + N - 1) * sizeof(double));
71  double *local_y = (double *)calloc(my_len, sizeof(double));
72  double *y       = NULL;
73  if (rank == 0)
74      y = (double *)calloc(L, sizeof(double));
75
76  if (!h || !local_x || !halo || !buf || !local_y ||
77      (rank == 0 && (!x_full || !y))) {
78      fprintf(stderr, "[rank_%d] Memory allocation failed.\n", rank);
79      MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
80  }
81
82  /* ----- */
83  /* Initialise */
84  /* ----- */
85  generate_fir_coefficients(h, N, 0.25);
86  if (rank == 0)
87      generate_signal(x_full, L, 42);
88
89  /* ----- */
90  /* Scatter signal using MPI_Scatterv (handles unequal chunks) */
91  /* ----- */
92  MPI_Scatterv(x_full, counts, displs, MPI_DOUBLE,
93              local_x, my_len, MPI_DOUBLE,
94              0, MPI_COMM_WORLD);
95
96  /* ----- */
97  /* Timed section: halo exchange + local filtering + gather */
98  /* ----- */
99  MPI_Barrier(MPI_COMM_WORLD);
100 double t_start = MPI_Wtime();
101

```

```

102  /*
103   * Halo exchange: each rank sends its last N-1 samples rightward
104   * and receives N-1 samples from its left neighbour.
105   * Guard against chunk smaller than halo (very high P, small L).
106   */
107  int halo_len = (my_len < N - 1) ? my_len : N - 1;
108
109  MPI_Sendrecv(
110      local_x + my_len - halo_len, halo_len, MPI_DOUBLE, right_rank, 0,
111      halo, halo_len, MPI_DOUBLE, left_rank, 0,
112      MPI_COMM_WORLD, MPI_STATUS_IGNORE
113  );
114
115  /* Build extended buffer: [halo | local_x] */
116  memcpy(buf, halo, halo_len * sizeof(double));
117  memcpy(buf + halo_len, local_x, my_len * sizeof(double));
118
119  /* Apply full N-tap filter to local chunk */
120  for (int n = halo_len; n < my_len + halo_len; n++) {
121      double acc = 0.0;
122      for (int k = 0; k <= halo_len; k++)
123          acc += h[k] * buf[n - k];
124      local_y[n - halo_len] = acc;
125  }
126
127  /* Gather results at rank 0 */
128  MPI_Gatherv(local_y, my_len, MPI_DOUBLE,
129              y, counts, displs, MPI_DOUBLE,
130              0, MPI_COMM_WORLD);
131
132  MPI_Barrier(MPI_COMM_WORLD);
133  double t_end = MPI_Wtime();
134
135  /* ----- */
136  /* Verification and report (rank 0 only) */
137  /* ----- */
138  if (rank == 0) {
139      char fname[64];
140      snprintf(fname, sizeof(fname), "ref_N%d_L%d.bin", N, L);
141      FILE *f = fopen(fname, "rb");
142      if (f) {
143          double *ref = (double *)malloc(L * sizeof(double));
144          fread(ref, sizeof(double), L, f);
145          fclose(f);
146          double __attribute__((unused)) err = verify(ref, y, L, N);
147          fprintf(stderr, "[parallel] max_abs_error = %.3e\n", err);
148          free(ref);
149      }
150      printf("%d %d %d %.6f\n", N, L, size, t_end - t_start);
151  }
152
153  free(h); free(local_x); free(halo); free(buf); free(local_y);
154  free(counts); free(displs);
155  if (rank == 0) { free(x_full); free(y); }
156  MPI_Finalize();
157  return EXIT_SUCCESS;
158 }

```

Listing 12: src/fir_parallel.c